

GOOGLE FDSE APPLIED AI III

Interview Preparation Plan and Progress Tracker

A focused six-week roadmap with an optional two-week buffer

Profile	Preparation target
Current baseline	Hiring Assessment passed (valid for 24 months for applications that require it); strong LeetCode foundation; Grokking System Design completed once; working full-time.
Weekly commitment	About 16–18 hours across six study days, with Friday protected for rest or optional light review.
Preparation pillars	DSA and coding • multilingual NLP • AI/ML system design • practical engineering • customer-facing judgment • behavioral stories.
Role lens	Prepare as a production-minded engineer who can deploy multilingual NLP and AI agent systems, harden prototypes for production and explain tradeoffs with customers and stakeholders.

How to use this tracker

Work through the weekly plan in order, but use the checkpoints to adjust emphasis. Check an item only when you can explain or perform it without notes. Keep a short mistake log and revisit it every Sunday. The exact interview rounds can vary by team and hiring process; prepare for the competencies rather than assuming a fixed loop.

Starting point: establish a timed DSA baseline and attempt “Design an enterprise RAG system for 10 million documents” from a blank page. Record what felt difficult before starting Week 1.

Your north star

Maintain DSA, refresh classical system design, become strong at AI system design, build comfort with practical coding, and become fluent in RAG, agents, LLM infrastructure, and customer delivery.

Weekly rhythm and time budget

Day	Time	Default block
Monday	2 h	60 min DSA + 60 min AI/system design
Tuesday	2 h	60 min DSA + 60 min AI/system design
Wednesday	2 h	60 min DSA + 60 min AI/system design
Thursday	2 h	60 min DSA + 60 min AI/system design
Friday	0–1 h	Off; optional light review only
Saturday	4–5 h	2 h AI design + 1.5 h practical coding + 1 h DSA
Sunday	4–5 h	2 h AI design + 1 h DSA + 1–2 h behavioral/review

A 16-hour week is the floor; 18 hours is the upper target. Keep one recovery day. If work gets heavy, preserve the weekly design practice and practical exercise, then reduce DSA volume rather than sacrificing sleep.

A repeatable study session

Block	What to do
DSA (60 min)	5 min restate constraints; 30–35 min solve; 10 min test edge cases; 10 min explain complexity and log the lesson.
AI design (60 min)	15 min learn/review one concept; 30 min draw or talk through architecture; 15 min list tradeoffs, failure modes, and open questions.
Weekend practical (90 min)	Clarify requirements, implement the smallest useful solution, add failure handling, and write a brief operational note.
Sunday review (30–45 min)	Revisit misses; select next week's weak spots; update hours, confidence, and story coverage.

Baseline and scorecard

Measure	Baseline	Weekly target / evidence
Timed DSA unfamiliar medium	____ min / hints ____	Reach a clean approach and tested code within 35–40 min; explain complexity.
RAG design from blank page	Confidence 1–5: ____	Complete a coherent 45–60 min design and name key tradeoffs.
Practical coding	Confidence 1–5: ____	Finish a 60–90 min exercise with validation and failure handling.
Behavioral stories	Ready: ____ / 8	Eight concise stories with personal actions and measurable outcomes.
Weekly hours	____	Target 16–18; record actual, not planned, hours.

Six-week roadmap at a glance

Week	Main DSA	AI / system design	Practical + people focus	Checkpoint
1	Arrays, hashing, windows, intervals, stacks, Trie	RAG foundations; classical design refresh	LRU Cache + GetRandom; first 2 stories	Explain end-to-end RAG and retrieval choices
2	Graphs, BFS/DFS, topo sort, Dijkstra, backtracking	Production RAG + multilingual NLP deployment and evaluation	Document pipeline; cross-language retrieval checks	Design multilingual enterprise RAG and troubleshoot failures
3	Bounded DP + greedy	LLM serving, latency, throughput, cost	Toy inference API; draft all 8 stories	Explain serving bottlenecks and DP states
4	Mixed timed Google-style practice	Agents, orchestration, state, tool reliability	Agent loop debugging; structured outputs	Trace a slow or incorrect agent end to end
5	Maintain strengths; targeted weak areas	Evaluation, guardrails, security, cloud reliability	Customer scenarios; mock rotations begin	Recommend tradeoffs under customer constraints
6	Interview simulations; no topic binge	Integrated AI design practice	Full simulation and weakness list	Ready across coding, design, practical, behavioral

Suggested allocation each week: roughly 6 hours DSA, 6–7 hours AI/system design, 2–3 hours practical coding, and 1–2 hours behavioral/review. The mix can shift based on checkpoint results.

WEEK 1 — Rebuild foundations

Get classical system design concepts fresh while starting RAG. Do not reread all of Grokking; refresh the concepts you need.

Day / block	Plan
Mon	Arrays + hash maps: Two Sum; Group Anagrams; Product of Array Except Self. RAG: purpose, RAG vs fine-tuning, offline vs online pipeline, parsing, chunking, embeddings.
Tue	Sliding window: Longest Substring Without Repeating Characters; Longest Repeating Character Replacement. Compare fixed, recursive, semantic, and structure-aware chunking.
Wed	Intervals: Merge Intervals; Non-overlapping Intervals; Meeting Rooms. Refresh load balancing, caching, databases, replication, sharding, queues, API gateway.
Thu	Monotonic stack/queue: Daily Temperatures; Next Greater Element; Sliding Window Maximum. Retrieval: embeddings, cosine similarity, ANN, HNSW, IVF/PQ conceptually.
Sat	Design enterprise RAG for 10M documents. Implement LRU Cache and Insert Delete GetRandom $O(1)$. Mixed DSA review.
Sun	BM25, inverted indexes, dense/hybrid retrieval, reranking. Implement Trie; Add and Search Word. Draft stories: biggest technical project; difficult/ambiguous problem.

Week checkpoint

☐ By Sunday, sketch ingestion → parsing → chunking → embedding → vector store → retrieval → reranking → context → LLM → validation/response. Explain why chunking and hybrid retrieval matter.

WEEK 2 — Graphs, multilingual NLP, and production RAG

Build graph pattern recognition and reason about multilingual quality, RAG failure modes, scale, freshness, and security.

Day / block	Plan
Mon	Number of Islands; Clone Graph. Review BFS/DFS and connected components. NLP deployment: language identification, language/script coverage, and data quality by language.
Tue	Course Schedule; Course Schedule II. Topological sort and cycle detection. NLP: normalization, tokenization, Unicode/script handling, code-switching, language-specific preprocessing, and safe fallbacks.
Wed	Rotting Oranges; Pacific Atlantic Water Flow. Multi-source BFS. RAG: multilingual embeddings, cross-lingual retrieval, BM25 vs dense vs hybrid.
Thu	Dijkstra; Network Delay Time. NLP evaluation: per-language quality slices, representative test sets, retrieval/answer metrics, regressions, and human review.
Sat	Build a simple document pipeline: parse multilingual mock docs, preserve language/tenant metadata, chunk, mock embeddings, retrieve. Compare same-language and cross-language queries.
Sun	Word Search; Word Search II; revisit Trie. Run a 45-minute multilingual enterprise RAG design and answer failure scenarios aloud.

Week checkpoint

☐ Checkpoint: design for 10M documents and explain a multilingual NLP deployment/evaluation plan. Diagnose bad retrieval, stale documents, language-specific quality gaps, <200 ms retrieval, growth, and strict tenant isolation.

WEEK 3 — Bounded DP and LLM serving

Cover high-value DP patterns without letting obscure Hard problems consume the week; understand serving architecture and metrics.

Day / block	Plan
Mon	House Robber; House Robber II. Serving path: client → gateway → router → model server/GPU → streaming response.
Tue	Coin Change; Partition Equal Subset Sum. Batching, continuous batching, KV cache, PagedAttention.
Wed	Word Break; Decode Ways. Parallelism: tensor vs pipeline; quantization; model choice.
Thu	LIS; LCS. Metrics: TTFT, TPOT/ITL, throughput, cost per request/token, GPU utilization.
Sat	Build toy POST /generate API with validation, timeout, retry policy, rate limiting, logs, streaming simulation. Review DP explanations.
Sun	Unique Paths; Minimum Path Sum; Stock with Cooldown; Stock with Transaction Fee. Draft all eight behavioral stories.

Week checkpoint

☐ Checkpoint: explain DP as state → transition → base case → iteration order → complexity. Diagnose high latency by separating queueing, prefill, decode, retrieval, and downstream time.

WEEK 4 — Agents and practical coding

Practice exact timed DSA problems, including stretch questions, and become fluent in tool calling, state, agent execution limits, and production debugging.

Day / block	Plan
Mon	DSA: Sliding Window Maximum (stretch) + re-solve Longest Substring Without Repeating Characters. AI: tool schemas and function calling.
Tue	DSA: Kth Largest Element + Top K Frequent Elements. AI: ReAct, tool selection, and tool-description quality.
Wed	DSA: Lowest Common Ancestor + tree traversal review. AI: state, context, and memory.
Thu	DSA: Largest Rectangle in Histogram (stretch) + Daily Temperatures. AI: retries, timeouts, idempotency, and execution budgets.
Sat	DSA: Word Search II (stretch) timed attempt + Trie review. Practical: debug an agent that repeats a tool call; trace request → model → decision → tool → latency/result → next step.
Sun	DSA: Network Delay Time timed re-solve; review the week's misses. Practical: implement structured-output validation with schema checks, bounded repair/retry, fallback, and clear error path. Rehearse ambiguous-problem story.

Week checkpoint

☐ Checkpoint: explain why an agent is slow or wrong from traces and propose a safe, bounded fix. Be able to discuss context-window management and tool permissions.

WEEK 5 — Reliability, evaluation, and customer engineering

Maintain coding fluency with assigned questions while turning architecture knowledge into operational and customer-facing judgment.

Day / block	Plan
Mon	DSA: Edit Distance (stretch) + re-solve Word Break. AI: offline/online evaluation, retrieval recall/precision, relevance, groundedness.
Tue	DSA: Find Median from Data Stream + Kth Largest re-solve. AI: correctness, latency, cost, feedback, escalation, abandonment.
Wed	DSA: Network Delay Time + Course Schedule II. AI: guardrails, JSON schema, prompt injection, data leakage, authorization, PII.
Thu	DSA: Gas Station + Jump Game II; explain why the greedy choice is safe. Infra: IAM, VPC, secrets, encryption, CMEK, tenant isolation.
Sat	DSA mock: 45 minutes on Minimum Window Substring (stretch), then review. Practical mock: messy nested JSON normalization or bounded concurrent API client. Customer role-play: “the system is hallucinating” or “the agent takes 40 seconds.”
Sun	DSA repair: re-solve the hardest miss from Mon–Sat without notes. Run one AI design mock and one customer/behavioral mock. Review tradeoffs and story clarity.

Week checkpoint

☐ Checkpoint: make recommendations from latency, quality, cost, security, and availability requirements. Explain LLM-as-judge limits; never treat a single judge score as ground truth.

WEEK 6 — Interview simulation

Consolidate with named mock questions. Stop starting large courses and simulate unfamiliar prompts under time limits.

Day / block	Plan
Mon	45-minute DSA mock: Median of Two Sorted Arrays (stretch). Focus on a correct invariant and clear progress; review afterward even if incomplete.
Tue	60-minute AI system-design mock: multilingual enterprise RAG with per-language quality targets, tenant isolation, and latency/cost constraints.
Wed	45–60-minute practical coding mock: implement a bounded-retry API client with timeout, rate limiting, and structured logs.
Thu	45-minute behavioral/customer mock: prototype-to-production failure plus a customer disagreement story.
Sat	Full practice block: 45 min DSA mock on Word Search II or Largest Rectangle in Histogram (pick the weaker one), 15 min break, 60 min AI design, 15 min break, 60 min practical engineering.
Sun	Review every miss. Re-solve one DSA miss from the week; write “My Interview Weakness List”; choose the top three fixes for buffer weeks or final review.

Week checkpoint

☐ Checkpoint: start unfamiliar tasks promptly, clarify constraints, communicate tradeoffs, test edge cases, and close with risks/next steps. Practice competencies, not a presumed exact interview sequence.

DSA question and pattern checklist

Use this as a bounded core set. For each problem, track: first solve, clean re-solve, timed re-solve, and explanation. A check means you can derive the approach and test it, not just recognize a memorized solution.

Arrays and hashing

Done	Problem / pattern	First	Clean	Timed
☐	Two Sum	☐	☐	☐
☐	3Sum	☐	☐	☐
☐	Group Anagrams	☐	☐	☐
☐	Product of Array Except Self	☐	☐	☐
☐	Top K Frequent Elements	☐	☐	☐
☐	Subarray Sum Equals K	☐	☐	☐

Sliding window

Done	Problem / pattern	First	Clean	Timed
☐	Longest Substring Without Repeating Characters	☐	☐	☐
☐	Longest Repeating Character Replacement	☐	☐	☐
☐	Minimum Window Substring	☐	☐	☐
☐	Sliding Window Maximum	☐	☐	☐

Intervals and greedy

Done	Problem / pattern	First	Clean	Timed
☐	Merge Intervals	☐	☐	☐
☐	Non-overlapping Intervals	☐	☐	☐
☐	Meeting Rooms	☐	☐	☐
☐	Meeting Rooms II	☐	☐	☐
☐	Gas Station	☐	☐	☐
☐	Jump Game	☐	☐	☐
☐	Jump Game II	☐	☐	☐
☐	Partition Labels	☐	☐	☐
☐	Task Scheduler	☐	☐	☐
☐	Assign Cookies	☐	☐	☐
☐	Minimum Number of Arrows	☐	☐	☐

Stacks and queues

Done	Problem / pattern	First	Clean	Timed
☐	Valid Parentheses	☐	☐	☐
☐	Daily Temperatures	☐	☐	☐
☐	Next Greater Element	☐	☐	☐
☐	Largest Rectangle in Histogram	☐	☐	☐
☐	Min/Max Stack	☐	☐	☐

Graphs and backtracking

Done	Problem / pattern	First	Clean	Timed
☐	Number of Islands	☐	☐	☐
☐	Clone Graph	☐	☐	☐
☐	Course Schedule	☐	☐	☐
☐	Course Schedule II	☐	☐	☐
☐	Rotting Oranges	☐	☐	☐
☐	Pacific Atlantic Water Flow	☐	☐	☐
☐	Network Delay Time	☐	☐	☐
☐	Word Search	☐	☐	☐
☐	Word Search II	☐	☐	☐

Trees

Done	Problem / pattern	First	Clean	Timed
☐	Maximum Depth	☐	☐	☐
☐	Diameter of Binary Tree	☐	☐	☐
☐	Level Order Traversal	☐	☐	☐
☐	Validate BST	☐	☐	☐
☐	Lowest Common Ancestor	☐	☐	☐

Heaps

Done	Problem / pattern	First	Clean	Timed
☐	Kth Largest Element	☐	☐	☐
☐	Top K Frequent Elements	☐	☐	☐
☐	Merge K Sorted Lists	☐	☐	☐
☐	Find Median from Data Stream	☐	☐	☐

Trie

Done	Problem / pattern	First	Clean	Timed
☐	Implement Trie	☐	☐	☐
☐	Add and Search Word	☐	☐	☐
☐	Word Search II	☐	☐	☐

Dynamic programming

Done	Problem / pattern	First	Clean	Timed
☐	Climbing Stairs	☐	☐	☐
☐	House Robber	☐	☐	☐
☐	House Robber II	☐	☐	☐
☐	Coin Change	☐	☐	☐
☐	Word Break	☐	☐	☐
☐	Decode Ways	☐	☐	☐
☐	Longest Increasing Subsequence	☐	☐	☐
☐	Unique Paths	☐	☐	☐
☐	Unique Paths II	☐	☐	☐
☐	Minimum Path Sum	☐	☐	☐
☐	Longest Common Subsequence	☐	☐	☐
☐	Edit Distance	☐	☐	☐

Done	Problem / pattern	First	Clean	Timed
☐	Partition Equal Subset Sum	☐	☐	☐
☐	Target Sum	☐	☐	☐
☐	Stock with Cooldown	☐	☐	☐
☐	Stock with Transaction Fee	☐	☐	☐

Data-structure design

Done	Problem / pattern	First	Clean	Timed
☐	LRU Cache	☐	☐	☐
☐	Insert Delete GetRandom O(1)	☐	☐	☐
☐	Time Based Key-Value Store	☐	☐	☐
☐	Design Twitter	☐	☐	☐
☐	Min/Max Stack	☐	☐	☐
☐	Implement Trie	☐	☐	☐

DSA interview habits

- ☐ State a brute-force approach before optimizing.
- ☐ Name the invariant or pattern and why it applies.
- ☐ Work through a small example and edge cases.
- ☐ Explain time and space complexity.
- ☐ For DP, state → transition → base case → order → complexity.
- ☐ For greedy, explain why the local choice is safe.
- ☐ Keep a mistake log: missed pattern, bug, edge case, or communication gap.

AI and ML system design syllabus

The aim is production reasoning, not memorizing vendor product names. For each topic, explain when to use it, what can fail, how to measure it, and which requirement changes the design.

RAG and enterprise knowledge

- ☐ End-to-end ingestion → parsing/OCR → chunking → embedding → vector DB → retrieval → reranking → context → LLM → validation/guardrails → response
- ☐ Batch vs streaming; metadata; incremental indexing; duplicate detection; freshness and deletion propagation
- ☐ Fixed-size, overlap, recursive, semantic, and structure-aware chunking; effect on retrieval and hallucination
- ☐ Dense retrieval, cosine/dot product, ANN, HNSW, IVF/PQ; sparse BM25/inverted index; hybrid fusion
- ☐ Reranking with cross-encoder: broad/cheap retrieval vs narrower/more precise ranking
- ☐ Context construction, metadata filters, citations, tenant isolation, ACL-aware retrieval

Multilingual NLP deployment and evaluation

- ☐ Language identification and language/script coverage; representative, consented, high-quality data by language
- ☐ Normalization, tokenization, Unicode/script handling, code-switching, transliteration, and language-specific preprocessing
- ☐ Model selection: multilingual model vs language-specific models; quality, latency, cost, and operational tradeoffs
- ☐ Cross-lingual and same-language retrieval; multilingual embeddings, BM25 limitations, hybrid retrieval, language-aware reranking
- ☐ Evaluation sliced by language, script, domain, and user journey; avoid aggregate scores hiding weak-language failures
- ☐ Fallback and escalation when language confidence is low; monitor drift, coverage, and regressions after model/data changes

LLM serving

- ☐ Request routing; batching and continuous batching; KV cache; PagedAttention
- ☐ GPU utilization, model selection, quantization, autoscaling, streaming
- ☐ Tensor vs pipeline parallelism; where complexity and communication overhead arise
- ☐ TTFT, TPOT/ITL, throughput, cost per request/token; p50/p95/p99 and queueing

Agents

- ☐ Tool/function calling and schemas; tool selection; ReAct; planning
- ☐ Agent state, conversation history, short- and long-term memory; context windows
- ☐ Multi-agent coordination and when it adds overhead
- ☐ Retries, timeouts, idempotency, loops, execution limits, tool failures, tracing

Evaluation, reliability, guardrails

- ☐ Offline golden sets; correctness, relevance, groundedness, retrieval recall/precision
- ☐ Online success, user feedback, escalation, hallucination signals, latency, tool failures, abandonment
- ☐ LLM-as-judge strengths and limitations; human review and calibration
- ☐ Structured output/schema validation; bounded retry/repair; fallback and human/error path
- ☐ Prompt injection, PII, content safety, authorization, least privilege, data leakage

Production infrastructure and security

- ☐ Containers, Kubernetes, load balancing, autoscaling, queues, Pub/Sub/Kafka concepts

- ☐ Redis, SQL/NoSQL, object storage; monitoring, tracing, alerting, regional failover
- ☐ IAM/service accounts, VPC/private networking, secrets, encryption, audit logs, CMEK conceptually
- ☐ Rate limiting, circuit breakers, backpressure, retries, graceful degradation, recovery

Design prompts to practice

- ☐ Enterprise RAG over 10 million documents.
- ☐ Customer-support agent with strict security and escalation requirements.
- ☐ Multi-agent research system; AI coding assistant; AI search engine.
- ☐ LLM inference platform; real-time conversational agent.
- ☐ Agent with 50 external tools; multimodal enterprise assistant.
- ☐ AI system with a strict 200 ms retrieval SLA or 1 million documents/day.
- ☐ Agent that occasionally chooses the wrong tool; RAG system with poor retrieval.
- ☐ LLM costs suddenly double; system must operate through a regional outage.

Universal AI design walkthrough

Step	Prompts to answer
1	Clarify users, core use cases, and success criteria.
2	Set scale, latency/quality/availability SLOs, freshness, security, and cost constraints.
3	Draw the high-level architecture and online/offline data flows.
4	Choose model, retrieval, agent, storage, and caching approach; justify alternatives.
5	Cover async work, retries, rate limits, failure isolation, fallbacks, and regional behavior.
6	Define IAM, tenant boundaries, privacy, prompt/tool protections, and auditability.
7	Specify tracing, evaluation, rollout, monitoring, and alerting.
8	Name bottlenecks, tradeoffs, risks, and the next measurement or experiment.

Practical coding exercise bank

Complete one 60–90 minute exercise each week. Keep the scope small and make behavior observable. Use the language you expect to interview in when practical.

Exercise	Core requirements	What good looks like
1. Resilient API client	Timeout, bounded retries, exponential backoff with jitter, rate limit, clear errors	Retries only safe/transient failures; respects deadline; logs request ID; tests edge cases.
2. Messy nested JSON	Parse customer records, normalize fields, validate, report invalid rows	Handles missing/null/wrong types; partial failures are explicit; summary counts reconcile.
3. Document pipeline	Parse mock docs, chunk, preserve metadata, mock embeddings, retrieve	Incremental update/delete path; deterministic tests; tenant filter before content exposure.
4. Bounded concurrency	Call multiple endpoints with max concurrency, timeout, cancellation	No unbounded task creation; per-item errors preserved; aggregate results stable.
5. Agent-loop debugger	Find repeated tool invocation or 40–90 second latency	Trace steps and durations; bound iterations; explain whether tool, model, retrieval, or retry dominates.
6. Streaming generation API	POST /generate; validate; simulate streaming; timeout, rate limit, logging	Clear response contract; cancellation and errors handled; avoid logging sensitive prompt data.
7. RAG evaluation script	Golden examples, retrieval metrics, answer scoring, latency/cost capture	Separates retrieval quality from answer quality; records regressions; judge score is not treated as truth.
8. Multilingual quality slice	Build a small language-tagged test set; compare same-language and cross-language retrieval/answers	Reports per-language metrics and sample counts; catches quality gaps hidden by overall averages.
8. Structured output validator	JSON schema, bounded repair/retry, fallback/error path	Rejects unsafe/malformed output; limits retries; records reason without exposing secrets.

Practical coding review checklist

- ☐ Did I clarify expected inputs, outputs, error behavior, and scale?
- ☐ Are retries bounded and safe? Are timeout and cancellation propagated?
- ☐ Can I explain concurrency limits, rate limiting, and idempotency?
- ☐ Do logs/traces help diagnose failures while protecting sensitive data?
- ☐ Did I add a small set of representative tests or examples?
- ☐ Can I state what I would change before production?

Behavioral and customer-facing story bank

For the major technical project or architectural decision story, include multilingual NLP evidence if it reflects your real experience. Be candid about the boundaries of that experience and connect adjacent work carefully.

Prepare eight real stories from your experience. Do not invent results; quantify impact only when you can support it. Keep a 90-second version and a deeper follow-up version for each.

Story slot	Prompt / evidence to capture	Ready
1	Ambiguous problem: how did you clarify the goal and reduce uncertainty?	☐
2	Production failure: how did you diagnose, communicate, recover, and prevent recurrence?	☐
3	Technical disagreement: how did you listen, compare evidence, and align?	☐
4	Major architectural decision: what options and tradeoffs did you evaluate?	☐
5	Cross-team work: how did you create shared ownership and unblock delivery?	☐
6	Ownership: what did you take responsibility for beyond your assigned task?	☐
7	Difficult deadline: what did you cut, sequence, or communicate?	☐
8	Requirements changed: how did you reset scope and preserve trust?	☐

Story structure

Element	Notes to prepare
Situation + stakes	Who was affected? What was at risk?
Problem + constraints	Ambiguity, deadline, scale, reliability, security, or conflicting needs.
Your actions	Use "I" for your decisions and contributions; explain diagnosis and communication.
Technical choice + tradeoff	Options considered; why one path fit the constraints; what you gave up.
Result	Outcome and evidence; include what did not go as expected.
Learning	What changed in your approach afterward?

Customer scenarios to rehearse

- ☐ The customer wants a technically unsuitable solution. Clarify the outcome, show evidence, offer viable options, and align on a decision.
- ☐ Requirements keep expanding. Reconfirm goals, prioritize, make scope and impact visible, and agree on a delivery sequence.
- ☐ The prototype works but production fails. Diagnose traffic, data, latency, reliability, security, and operational gaps before promising a fix.
- ☐ The customer asks for 99.99% availability with little budget. Quantify the tradeoff and propose staged options.
- ☐ The elegant solution is six months away. Offer a safe incremental path with explicit limitations and migration plan.
- ☐ The customer says the model is hallucinating. Ask for examples, trace retrieval/context/model behavior, quantify, mitigate, and measure.
- ☐ The customer blames the system for an outage. Acknowledge impact, establish facts, coordinate recovery, communicate cadence, and follow through.

A useful response sequence: clarify → diagnose → options → tradeoffs → recommendation → alignment → execution. Demonstrate calm ownership and customer trust; avoid jumping immediately to a product or model choice.

Mock interview schedule and review rubric

Week	Mock plan	Review artifact
1	Baseline: 35–40 min unfamiliar medium; 45 min enterprise RAG sketch.	Baseline scorecard and top five knowledge gaps.
2	45 min graph session; 45–60 min multilingual enterprise RAG design.	Architecture sketch, per-language evaluation plan, and failure-mode checklist.
3	45 min DP session; 45 min LLM serving design; 30 min API exercise under time.	DP pattern errors; serving bottleneck notes.
4	45 min mixed DSA; 45 min agent reliability/debugging; 30 min customer scenario.	Agent trace diagnosis and story feedback.
5	One DSA mock, one AI design mock, one practical mock, one customer/behavioral mock across the week.	One-page rubric and prioritized fixes.
6	Saturday integrated practice block (45 DSA + 60 design + 60 practical, with breaks); Sunday review.	Final weakness list and confidence ratings.
7–8 optional	Two or three complete mocks total; keep a sustainable cadence, not daily full loops.	Focus on top recurring gaps; protect recovery time.

Score each mock from 1 to 4

Dimension	1 — needs work	2 — developing	3 — solid	4 — strong
Structure	Unclear start	Some structure, gaps	Clear sequence	Adapts structure naturally
Technical depth	Unsupported choices	Basic concepts	Sound tradeoffs	Anticipates failure and scale
Communication	Hard to follow	Inconsistent narration	Concise and collaborative	Clear, responsive, checks alignment
Execution	No validation	Some testing	Tests edge cases	Systematic tests and recovery
Customer judgment	Jumps to solution	Limited tradeoffs	Clarifies and aligns	Builds trust under uncertainty

After each mock

- ☐ Write one thing to keep doing.
- ☐ Write the two most important improvements.
- ☐ Classify each miss: knowledge, pattern recognition, implementation, testing, communication, or judgment.
- ☐ Schedule a short repair session within the next 48 hours.

Progress tracker and checkpoints

Week	Hours planned / actual	DSA confidence 1–5	AI design 1–5	Practical 1–5	Story / customer 1–5	Top fix / notes
1	___ / ___	___	___	___	___	___
2	___ / ___	___	___	___	___	___
3	___ / ___	___	___	___	___	___
4	___ / ___	___	___	___	___	___
5	___ / ___	___	___	___	___	___
6	___ / ___	___	___	___	___	___
7	___ / ___	___	___	___	___	___
8	___ / ___	___	___	___	___	___

Checkpoint gates

Gate	Evidence to move on
End Week 1	Can explain a complete RAG pipeline, chunking options, dense vs sparse retrieval, and core system design refresh.
End Week 2	Can design for scale/freshness/tenant isolation, outline multilingual NLP deployment and per-language evaluation, and solve common BFS/DFS/topological patterns.
End Week 3	Can reason about serving latency and cost; explain core bounded DP patterns.
End Week 4	Can trace agent behavior and bound loops/retries; can implement structured output validation.
End Week 5	Can propose evaluation, security, reliability and customer-aligned tradeoffs.
End Week 6	Can complete a timed simulation and explain misses with a short improvement plan.

My Interview Weakness List

- ☐ _____
- ☐ _____
- ☐ _____
- ☐ _____
- ☐ _____

Optional Weeks 7–8 buffer

Use the buffer if the interview date is later or a checkpoint is still weak. Aim for 30% DSA, 40% system design, 20% practical coding, and 10% behavioral/customer practice. Run two or three complete mocks across the two weeks. Week 8 is interview mode: consolidate, sleep, and avoid starting a new course or chasing a large new problem set.

What not to do

- ☐ Do not make DP and Greedy the entire plan. Keep them bounded; prioritize clean pattern recognition and explanation.
- ☐ Do not restart all of Grokking System Design. Refresh the primitives and apply them to AI workloads.
- ☐ Do not pursue every obscure Hard DP problem. Use the scoped list and move on once you can explain state and transitions.
- ☐ Do not memorize vector database names or cloud product catalogs. Explain requirements, measurements, and tradeoffs first.
- ☐ Do not prepare as an ML researcher from scratch. Focus on production AI systems, practical engineering, and customer delivery.
- ☐ Do not turn Friday into a mandatory study day or target 30 hours a week while working full-time. Consistency and recovery matter.
- ☐ Do not build a real LLM as a prerequisite for practical exercises. A mock inference layer is enough to practice production behavior.
- ☐ Do not treat one fixed interview-loop structure as guaranteed. Exact rounds vary; follow recruiter guidance and prepare the competencies.
- ☐ Do not respond to “hallucination” or “slow” with a guess. Ask for examples, inspect traces and metrics, then isolate the failure.
- ☐ Do not start a new course in the final week. Use mock feedback to repair a few specific weaknesses.

Fast reference: FDSE thinking

Before choosing a technology, clarify the customer’s latency, accuracy, corpus size, freshness, security, availability, and cost requirements. Separate ingestion from online serving when useful; measure retrieval and answer quality independently; design for failure and observability; explain the tradeoff and align on the next step.

Interview readiness statement

I can solve an unfamiliar coding problem under time pressure; explain truthful evidence of multilingual NLP work; design production RAG, agent, and serving systems; implement practical reliability patterns; and work through customer ambiguity with calm, evidence-based tradeoffs.

Personal readiness rating (1–5): _____ Date: _____ Next review: _____